

SESSION 3

22.09.2024

Agenda

Lab 3 Use Pandas to load a CSV file, display the first 10 rows, and perform basic data manipulation (sorting, filtering).

Module-2 Data Analysis and Visualization with Python

Data Loading and Cleaning: Reading and Writing Data with Pandas,

Data Manipulation with Pandas,

Exploratory Data Analysis (EDA): Descriptive Statistics,

Data Visualization with Matplotlib and Seaborn,

Correlation and Covariance, Advanced Data Visualization-Interactive Plots with Plotly.

Lab 4 Load a dataset into a Pandas DataFrame and clean the data (handle missing values, drop unnecessary columns) and Perform group by operations and calculate summary statistics.

Lab 5 Create a variety of plots using Matplotlib, Use Seaborn to create advanced visualizations (box plot, heatmap) and customize them (titles, labels, colors).

Lab 3 Use Pandas to load and read a CSV file

Command to load csv file in google colab:

```
from google.colab import files
```

```
uploaded = files.upload()
```

For any notebook or Python IDLE, keep the dataset(csv file) in the folder where you saved your pythonfile, it will link automatically.

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
print(df.to_string())
```

Print the DataFrame without the `to_string()` method:

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
print(df)
```

max_rows

The number of rows returned is defined in Pandas option settings.

```
import pandas as pd

print(pd.options.display.max_rows)
```

Increase the maximum number of rows to display the entire DataFrame:

```
import pandas as pd

pd.options.display.max_rows = 9999

df = pd.read_csv('data.csv')

print(df)
```

Viewing the Data

One of the most used method for getting a quick overview of the DataFrame, is the `head()` method.

The `head()` method returns the headers and a specified number of rows, starting from the top.

Example: Printing the first 10 rows of the DataFrame:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head(10))
```

if the number of rows is not specified, the `head()` method will return the top 5 rows

Print the first 5 rows of the DataFrame:

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
print(df.head())
```

Print the last 5 rows of the DataFrame:

`tail()` method for viewing the *last* rows of the DataFrame.

The `tail()` method returns the specified number of rows, starting from the bottom.

```
print(df.tail())
```

Info About the Data

The DataFrames object has a method called `info()`, that gives you more information about the data set.
`print(df.info())`

Output: <class 'pandas.core.frame.DataFrame'>

RangeIndex: 169 entries, 0 to 168

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

---	-----	-----	-----
-----	-------	-------	-------

0	Duration	169 non-null	int64
---	----------	--------------	-------

1	Pulse	169 non-null	int64
---	-------	--------------	-------

2	Maxpulse	169 non-null	int64
---	----------	--------------	-------

3	Calories	164 non-null	float64
---	----------	--------------	---------

dtypes: float64(1), int64(3)

memory usage: 5.4 KB

None

The result tells us there are 169 rows and 4 columns:

And the name of each column, with the data type:

The **info()** method also tells us how many Non-Null values there are present in each column, and in our data set it seems like there are 164 of 169 Non-Null values in the "Calories" column.

Which means that there are 5 rows with no value at all, in the "Calories" column, for whatever reason.

Empty values, or Null values, can be bad when analyzing data, and you should consider removing rows with empty values. This is a step towards what is called *cleaning data*

Sorting Data Frames in Pandas

Create a [Pandas Dataframe](#) for Demonstration, in which we will perform various sorting functions.

```
import pandas as pd
```

```
# creating and initializing a nested list
```

```
age_list = [['Afghanistan', 1952, 8425333, 'Asia'],  
            ['Australia', 1957, 9712569, 'Oceania'],  
            ['Brazil', 1962, 76039390, 'Americas'],  
            ['China', 1957, 637408000, 'Asia'],  
            ['France', 1957, 44310863, 'Europe'],  
            ['India', 1952, 3.72e+08, 'Asia'],  
            ['United States', 1957, 171984000, 'Americas']]
```

```
# creating a pandas dataframe
```

```
df = pd.DataFrame(age_list, columns=['Country', 'Year',  
                                     'Population', 'Continent'])
```

```
df
```

Output

	Country	Year	Population	Continent
0	Afghanistan	1952	8425333.0	Asia
1	Australia	1957	9712569.0	Oceania
2	Brazil	1962	76039390.0	Americas
3	China	1957	637408000.0	Asia
4	France	1957	44310863.0	Europe
5	India	1952	372000000.0	Asia
6	United States	1957	171984000.0	Americas

The function `sort_values()` can sort the data frame in Ascending or Descending order.

Pandas DataFrame Sorting in Ascending Order

The code snippet sorts the DataFrame `df` in ascending order based on the 'Country' column.

```
# Sorting by column 'Country'
```

```
df.sort_values(by=['Country'])
```

Output:

	Country	Year	Population	Continent
0	Afghanistan	1952	8425333.0	Asia
1	Australia	1957	9712569.0	Oceania
2	Brazil	1962	76039390.0	Americas
3	China	1957	637408000.0	Asia
4	France	1957	44310863.0	Europe
5	India	1952	372000000.0	Asia
6	United States	1957	171984000.0	Americas

Sorting the Pandas DataFrame in Descending order

The DataFrame df will be sorted in descending order based on the “Population” column, with the country having the highest population appearing at the top of the DataFrame.

```
# Sorting by column "Population"
```

```
df.sort_values(by=['Population'],
```

Output:

	Country	Year	Population	Continent
3	China	1957	637408000.0	Asia
5	India	1952	372000000.0	Asia
6	United States	1957	171984000.0	Americas
2	Brazil	1962	76039390.0	Americas
4	France	1957	44310863.0	Europe
1	Australia	1957	9712569.0	Oceania
0	Afghanistan	1952	8425333.0	Asia

Sort Pandas DataFrame Based on Sampling

Here, we are sorting a DataFrame (df) based on the 'Population' column, arranging rows with missing values in 'Population' to appear first. The `sort_values()` method with the `na_position='first'` argument achieves this, prioritizing rows with missing values at the beginning of the sorted DataFrame.

```
# Sorting by column "Population"
```

```
# by putting missing values first
```

```
df.sort_values(by=['Population'], na_position='first')
```

Output:

	Country	Year	Population	Continent
6	United States	1957	0.0	Americas
0	Afghanistan	1952	8425333.0	Asia
1	Australia	1957	9712569.0	Oceania
4	France	1957	44310863.0	Europe
2	Brazil	1962	76039390.0	Americas
5	India	1952	372000000.0	Asia
3	China	1957	637408000.0	Asia

Sorting Data frames by Multiple Columns

In this example, we are sorting a DataFrame (df) primarily by the 'Country' column in ascending order and, within each country group, by the 'Continent' column. The resulting DataFrame is sorted based on the specified column order, creating a sorted dataset.

Output:

	Country	Year	Population	Continent
0	Afghanistan	1952	8425333.0	Asia
1	Australia	1957	9712569.0	Oceania
2	Brazil	1962	76039390.0	Americas
3	China	1957	637408000.0	Asia
4	France	1957	44310863.0	Europe
5	India	1952	372000000.0	Asia
6	United States	1957	171984000.0	Americas

Sorting Data frames by Columns But in a Different Order

In this example, we are sorting a DataFrame (df) first by the 'Country' column in descending order and, within each country group, by the 'Continent' column in ascending order. The resulting DataFrame is organized based on the specified column sorting criteria.

```
# Sorting by columns "Country" in descending
```

```
# order and then "Continent" in ascending order
```

```
df.sort_values(by=['Country', 'Continent'],ascending=[False, True])
```

Output:

	Country	Year	Population	Continent
6	United States	1957	171984000.0	Americas
5	India	1952	372000000.0	Asia
4	France	1957	44310863.0	Europe
3	China	1957	637408000.0	Asia
2	Brazil	1962	76039390.0	Americas
1	Australia	1957	9712569.0	Oceania
0	Afghanistan	1952	8425333.0	Asia

Filter Pandas Dataframe

Filtering a [Pandas DataFrame](#) by way of column values is a commonplace operation while running with information in [Python](#). You can use various methods and techniques for Pandas filtering. Here are numerous ways to filter out a Pandas DataFrame through column values.

- **Pandas Filtering using Exact Value**
- **Pandas Filtering based on Comparison Operator**
- **Filter Pandas DataFrame by List of Values**
- **Filter a DataFrame by a list of Strings**
- **Pandas Dataframe filter Multiple Conditions**
- **Filter Pandas Dataframe by Column Value Operator**

Pandas Filtering using Exact Value

```
# Filter rows where column 'A' is equal to 3
```

```
filtered_df = df.query('A == 3')
```

```
print(filtered_df)
```

Create a dataframe:

```
# importing pandas

import pandas as pd

# declare a dictionary

record = { 'Name' : ['Ankit', 'Swapnil', 'Aishwarya', 'Priyanka', 'Shivangi', 'Shaurya' ],
           'Age' : [22, 20, 21, 19, 18, 22],
           'Stream' : ['Math', 'Commerce', 'Science', 'Math', 'Math', 'Science'],
           'Percentage' : [90, 90, 96, 75, 70, 80] }

# create a dataframe

dataframe = pd.DataFrame(record,
                          columns = ['Name', 'Age',
                                     'Stream', 'Percentage'])

# show the Dataframe

print("Given Dataframe :\n", dataframe)
```

Output

Given Dataframe :

	Name	Age	Stream	Percentage
0	Ankit	22	Math	90
1	Swapnil	20	Commerce	90
2	Aishwarya	21	Science	96
3	Priyanka	19	Math	75
4	Shivangi	18	Math	70
5	Shaurya	22	Science	80

Pandas Filtering based on Comparison Operator

Example 1: Selecting all the rows from the given Dataframe in which 'Percentage' is greater than 75 using `[]`.

```
# selecting rows based on condition
```

```
rslt_df = dataframe[dataframe['Percentage'] > 70]
```

```
print('\nResult dataframe :\n', rslt_df)
```

Output

Result dataframe :

	Name	Age	Stream	Percentage
0	Ankit	22	Math	90
1	Swapnil	20	Commerce	90
2	Aishwarya	21	Science	96
3	Priyanka	19	Math	75
5	Shaurya	22	Science	80

using `loc[]`:

Selecting all the rows from the given Dataframe in which 'Percentage' is greater than 70 using `loc[]`.

selecting rows based on condition

```
rslt_df = dataframe.loc[dataframe['Percentage'] > 70]
```

```
print('\nResult dataframe :\n',rslt_df)
```

Output

Result dataframe :

	Name	Age	Stream	Percentage
0	Ankit	22	Math	90
1	Swapnil	20	Commerce	90
2	Aishwarya	21	Science	96
3	Priyanka	19	Math	75
5	Shaurya	22	Science	80

Filter Pandas DataFrame by List of Values

Selecting those rows of Pandas Dataframe whose column value is present in the list using [isin\(\)](#) method of the dataframe.

```
options = [22, 29]
```

```
# selecting rows based on condition
```

```
rslt_df = dataframe[dataframe['Age'].isin(options)]
```

```
print('\nResult dataframe :\n', rslt_df)
```

Output

```
Result dataframe :
```

	Name	Age	Stream	Percentage
0	Ankit	22	Math	90
5	Shaurya	22	Science	80

Filter a DataFrame by a list of Strings

Selecting all the rows from the given dataframe in which 'Stream' is present in the options list using [loc\[\]](#).

```
options = ['Science', 'Commerce']
```

```
# selecting rows based on condition
```

```
rslt_df = dataframe.loc[dataframe['Stream'].isin(options)]
```

```
print('\nResult dataframe :\n', rslt_df)
```

Output:

Result dataframe :

	Name	Age	Stream	Percentage
1	Swapnil	20	Commerce	90
2	Aishwarya	21	Science	96
5	Shaurya	22	Science	80

Pandas Dataframe filter Multiple Conditions

Selecting rows of Pandas Dataframe based on multiple column conditions using '&' operator.

```
options = ['Commerce', 'Science']
```

```
# selecting rows based on condition
```

```
rslt_df = dataframe[(dataframe['Age'] == 22) & dataframe['Stream'].isin(options)]
```

```
print('\nResult dataframe :\n', rslt_df)
```

Output:

```
Result dataframe :
```

	Name	Age	Stream	Percentage
5	Shaurya	22	Science	80

Filter Pandas Dataframe by Column Value Operator Chaining

Selecting all the rows from the given Dataframe in which 'Age' is equal to 22 and 'Stream' is present in the options list using [loc\[\]](#).

```
options = ['Commerce', 'Science']
```

```
# selecting rows based on condition
```

```
rslt_df = dataframe.loc[(dataframe['Age'] == 22) & dataframe['Stream'].isin(options)]
```

```
print('\nResult dataframe :\n', rslt_df)
```

Output:

```
Result dataframe :
```

	Name	Age	Stream	Percentage
5	Shaurya	22	Science	80

Pandas - Cleaning Data

Data cleaning means fixing bad data in your data set.

Bad data could be:

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates

Consider the dataset data.csv

The data set contains some empty cells ("Date" in row 22, and "Calories" in row 18 and 28).

The data set contains wrong format ("Date" in row 26).

The data set contains wrong data ("Duration" in row 7).

The data set contains duplicates (row 11 and 12).

Pandas - Cleaning Empty Cells

Empty Cells

Empty cells can potentially give you a wrong result when you analyze data.

Remove Rows

One way to deal with empty cells is to remove rows that contain empty cells.

This is usually OK, since data sets can be very big, and removing a few rows will not have a big impact on the result.

dropna():

By default, the **dropna()** method returns a *new* DataFrame, and will not change the original.

Return a new Data Frame with no empty cells.

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
new_df = df.dropna()
```

```
print(new_df.to_string())
```

If you want to change the original DataFrame, use the **inplace = True** argument:

Example: Remove all rows with NULL values:

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
df.dropna(inplace = True)
```

```
print(df.to_string())
```

Now, the **dropna(inplace = True)** will NOT return a new DataFrame, but it will remove all rows containing NULL values from the original DataFrame.

Replace Empty Values

Another way of dealing with empty cells is to insert a *new* value instead.

This way you do not have to delete entire rows just because of some empty cells.

The **fillna()** method allows us to replace empty cells with a value:

Example: Replace NULL values with the number 130:

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
df.fillna(130, inplace = True)
```


Replace Only For Specified Columns

The example above replaces all empty cells in the whole Data Frame.

To only replace empty values for one column, specify the *column name* for the DataFrame:

Example: Replace NULL values in the **"Calories" columns** with the number 130:

```
import pandas as pd  
df = pd.read_csv('data.csv')  
df["Calories"].fillna(130, inplace = True)
```

Replace Using Mean, Median, or Mode

A common way to replace empty cells, is to calculate the mean, median or mode value of the column.

Pandas uses the **mean()** **median()** and **mode()** methods to calculate the respective values for a specified column:

Example

Calculate the MEAN, and replace any empty values with it: [Mean = the average value (the sum of all values divided by number of values)].

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
x = df["Calories"].mean()
```

```
df["Calories"].fillna(x, inplace = True)
```

Example:

Calculate the MEDIAN, and replace any empty values with it: [Median = the value in the middle, after you have sorted all values ascending.]

```
x = df["Calories"].median()
```

```
df["Calories"].fillna(x, inplace = True)
```

Example:

Calculate the MODE, and replace any empty values with it: [Mode = the value that appears most frequently.]

```
x = df["Calories"].mode()[0]
```

```
df["Calories"].fillna(x, inplace = True)
```

Pandas - Cleaning Data of Wrong Format

Data of Wrong Format

Cells with data of wrong format can make it difficult, or even impossible, to analyze data.

To fix it, you have two options: remove the rows, or convert all cells in the columns into the same format.

Convert Into a Correct Format

In our Data Frame, we have two cells with the wrong format. Check out row 22 and 26, the 'Date' column should be a string that represents a date:

To convert all cells in the 'Date' column into dates.

Pandas has a `to_datetime()` method for this:

Example Convert to date:

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
print(df.to_string())
```

See from the result, the date in row 26 was fixed, but the empty date in row 22 got a NaT (Not a Time) value, in other words an empty value. One way to deal with empty values is simply removing the entire row.

21	60	'2020/12/21'	108	131	364.2
22	45	NaT	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	'2020/12/26'	100	120	250.0
27	60	'2020/12/27'	92	118	241.0

The date in row 26 was fixed, but the empty date in row 22 got a NaT (Not a Time) value, in other words an empty value. One way to deal with empty values is simply removing the entire row.

Removing Rows

The result from the converting in the example above gave us a NaT value, which can be handled as a NULL value, and we can remove the row by using the **dropna()** method.

Example Remove rows with a NULL value in the "Date" column:

```
df.dropna(subset=['Date'], inplace = True)
```

Pandas - Fixing Wrong Data

Wrong Data

"Wrong data" does not have to be "empty cells" or "wrong format", it can just be wrong, like if someone registered "199" instead of "1.99".

Sometimes you can spot wrong data by looking at the data set, because you have an expectation of what it should be.

If you take a look at our data set, you can see that in row 7, the duration is 450, but for all the other rows the duration is between 30 and 60.

It doesn't have to be wrong, but taking in consideration that this is the data set of someone's workout sessions, we conclude with the fact that this person did not work out in 450 minutes.

7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0

How can we fix wrong values, like the one for "Duration" in row 7?

Replacing Values

One way to fix wrong values is to replace them with something else.

In our example, it is most likely a typo, and the value should be "45" instead of "450", and we could just insert "45" in row 7:

Example Set "Duration" = 45 in row 7:

```
df.loc[7, 'Duration'] = 45
```

For small data sets you might be able to replace the wrong data one by one, but not for big data sets.

To replace wrong data for larger data sets you can create some rules, e.g. set some boundaries for legal values, and replace any values that are outside of the boundaries.

Example

Loop through all values in the "Duration" column.

If the value is higher than 120, set it to 120:

Example

Loop through all values in the "Duration" column.

If the value is higher than 120, set it to 120:

```
for x in df.index:
```

```
    if df.loc[x, "Duration"] > 120:
```

```
        df.loc[x, "Duration"] = 120
```

Removing Rows

Another way of handling wrong data is to remove the rows that contains wrong data.

Example

Delete rows where "Duration" is higher than 120:

```
for x in df.index:
```

```
    if df.loc[x, "Duration"] > 120:
```

```
        df.drop(x, inplace = True)
```

Pandas - Removing Duplicates

Discovering Duplicates

Duplicate rows are rows that have been registered more than one time.

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3

By taking a look at our test data set, we can assume that row 11 and 12 are duplicates.

To discover duplicates, we can use the `uplicated()` method.

The `uplicated()` method returns a Boolean values for each row:

Example

Returns `True` for every row that is a duplicate, otherwise `False`:

```
print(df.duplicated())
```

Example: Remove all duplicates:

```
df.drop_duplicates(inplace = True)
```

Note: The `(inplace = True)` will make sure that the method does NOT return a *new* DataFrame, but it will remove all duplicates from the *original* DataFrame.

Data Manipulation in Python using Pandas

In Machine Learning, the model requires a dataset to operate, i.e. to train and test.

But data doesn't come fully prepared and ready to use.

There are discrepancies like Nan/ Null / NA values in many rows and columns.

Sometimes the data set also contains some of the rows and columns which are not even required in the operation of our model.

In such conditions, it requires proper cleaning and modification of the data set to make it an efficient input for our model.

This is achieved by practicing **Data Wrangling** before giving data input to the model.

Output:

Creating Sample DataFrame

```
# Importing the pandas library

import pandas as pd

# creating a dataframe object

student_register = pd.DataFrame()

# assigning values to the

# rows and columns of the dataframe

student_register['Name'] = ['Abhijit','Smriti','Akash', 'Roshni']

student_register['Age'] = [20, 19, 20, 14]

student_register['Student'] = [False, True,True, False]

print(student_register)
```

	Name	Age	Student
0	Abhijit	20	False
1	Smriti	19	True
2	Akash	20	True
3	Roshni	14	False

As you can see, the dataframe object has four rows [0, 1, 2, 3] and three columns["Name", "Age", "Student"] respectively.

The column which contains the index values i.e. [0, 1, 2, 3] is known as the **index column**, which is a default part in pandas datagram.

We can change that as per our requirement too because Python is powerful.

Adding data in DataFrame using Append Function

To add a new student in the datagram, i.e you want to add a new row to your existing data frame, that can be achieved by the following code snippet.

One important concept is that the “dataframe” object of Python, consists of rows which are “series” objects instead, stack together to form a table.

Hence adding a new row means creating a new series object and appending it to the dataframe.

```
# creating a new pandas
```

```
# series object
```

```
new_person = pd.Series(['Mansi', 19, True],
```

```
index = ['Name', 'Age', 'Student'])
```

```
# using the .append() function # to add that row to the dataframe
```

```
student_register.append(new_person, ignore_index = True)
```

```
print(student_register)
```


Output:

	Name	Age	Student
0	Abhijit	20	False
1	Smriti	19	True
2	Akash	20	True
3	Roshni	14	False

Data Manipulation on Dataset

How to perform dataframe creation and data addition in a big dataset.

To exact information of each column, i.e. what type of value it stores and how many of them are unique. There are three support functions, **.shape**, **.info()** and **.corr()** which output the shape of the table, information on rows and columns, and correlation between numerical columns.

```
# dimension of the dataframe
```

```
print('Shape: ')
```

```
print(student_register.shape)
```

```
# showing info about the data
```

```
print('Info: ')
```

```
print(student_register.info())
```

```
# correlation between columns
```

```
print('Correlation: ')
```

```
print(student_register.corr())
```

Shape:

(4, 3)

Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 4 entries, 0 to 3

Data columns (total 3 columns):

Column Non-Null Count Dtype

--- ---

0 Name 4 non-null object

1 Age 4 non-null int64

2 Student 4 non-null bool

dtypes: bool(1), int64(1), object(1)

memory usage: 196.0+ bytes

None

Correlation:

Age Student

1 -0.000000 0.500000

In the above example, the **.shape** function gives an output (4, 3) as that is the size of the created dataframe.

The description of the output given by .info() method is as follows:

1. **RangeIndex** describes about the index column, i.e. [0, 1, 2, 3] in our datagram. Which is the number of rows in our dataframe.
2. As the name suggests **Data columns** give the total number of columns as output.
3. **Name, Age, Student** are the name of the columns in our data, non-null tells us that in the corresponding column, there is no NA/ Nan/ None value exists. **object, int64** and **bool** are the datatypes each column have.
4. **dtype** gives you an overview of how many data types present in the datagram, which in term simplifies the data cleaning process.

Also, in high-end machine learning models, **memory usage** is an important term, we can't neglect that.

Getting Statistical Analysis of Data

Before processing and wrangling any data you need to get the overview of it, which includes statistical conclusions like **standard deviation(std), mean and it's quartile distributions**.

```
# for showing the statistical
```

```
# info of the dataframe
```

```
print('Describe')
```

```
print(student_register.describe())
```

Output:

Describe

Age

count 4.000000

mean 18.250000

std 2.872281

min 14.000000

25% 17.750000

50% 19.500000

75% 20.000000

max 20.000000

The description of the output given by `.describe()` method is as follows:

1. **count** is the number of rows in the dataframe.
2. **mean** is the mean value of all the entries in the “Age” column.
3. **std** is the standard deviation of the corresponding column.
4. **min** and **max** are the minimum and maximum entry in the column respectively.
5. 25%, 50% and 75% are the **First Quartiles**, **Second Quartile(Median)** and **Third Quartile** respectively, which gives us important info on the distribution of the dataset and makes it simpler to apply an ML model.

Dropping Columns from Data

Drop a column from the data. We will use the drop function from the pandas. We will keep axis = 1 for columns.

```
students = student_register.drop('Age', axis=1)
```

```
print(students.head())
```

Output:

	Name	Student
0	Abhijit	False
1	Smriti	True
2	Akash	True
3	Roshni	False

From the output, we can see that the 'Age' column is dropped.

Dropping Rows from Data

To Drop a row from the dataset, use drop function. We will keep axis=0.

```
students = students.drop(2, axis=0)
```

```
print(students.head())
```

Output:

	Name	Student
0	Abhijit	False
1	Smriti	True
3	Roshni	False

In the output we can see that the 2 row is dropped.